Product Handbook

Online Boutique (microservices-demo)

Current-state reference for the AI Change Impact Analyzer

Capstone Project — Team 7

Companion document to Historical & Contextual Data

Source of truth: GoogleCloudPlatform/microservices-demo, branch main

Every factual claim in this handbook is cross-referenced in Appendix A.

1. Product Overview

Online Boutique is a cloud-first e-commerce demo application. End users browse a small catalog of products, add items to a shopping cart, and complete a checkout flow that includes currency conversion, shipping quoting, and a mock payment. The application is intentionally simple as a product but architecturally non-trivial — it is composed of 11 microservices in 5 languages communicating over gRPC.

It is operated by Google as a reference application for Kubernetes, gRPC, service mesh, and observability technologies. There are no real customers, no real payments, and no real shipments. For the purposes of the AI Change Impact Analyzer, we treat it as if it were a real production system: changes have impact, services have contracts, and breaking those contracts has consequences.

1.1 Scope of this handbook

This document describes the current state of the system as found in branch main of the upstream repository. It captures what the system does, what its public and internal contracts are, what it is configured with, and where its known weak spots are. It does NOT cover:

- History of changes — that is in the companion Historical & Contextual Data document
- Operational dashboards, alerts, or runbooks — those are partly in the historical document and partly in the live monitoring system
- Deployment and CI/CD pipelines — those are infrastructure, not product

1.2 How the AI Change Analyzer uses this document

When a proposed change arrives, the analyzer consults the handbook first to answer: what is the current state of the thing being changed? Only with that anchor in place can it then ask: what historical incidents resemble this? what does the dependency graph say? what anti-patterns apply?

Each section is structured to be retrievable as a coherent chunk. The data models, API contracts, and configuration reference are the highest-value retrievals — they let the analyzer reason about what a diff actually means.

2. Feature Inventory

The user-facing product surface, broken down into features and their current state. Status reflects what is in the main branch as of the document date.

Feature	Status	Description
Browse catalog (home)	GA	Anonymous users see a grid of products on the home page.
Product detail page	GA	Per-product page with description, price, and an Add-to-Cart action.
Search products	GA	Backed by SearchProducts RPC; substring search over name/description.
Shopping cart	GA	Add items, view cart, empty cart. Cart is keyed to an auto-generated session ID; no signup or login.
Currency selection	GA	User picks a display currency; all prices on every page are converted via CurrencyService.
Shipping quote	GA	Quote shown on the checkout page based on cart contents and entered address.
Place order	GA	Single-step checkout: collects email, address, credit card; runs payment, ships, and emails a confirmation.
Recommendations	GA	Each product page and the cart show a 'You may also like' strip from RecommendationService.
Text ads	GA	Contextual text ads on certain pages, served by AdService based on page keywords.
AI Shopping Assistant	Optional	Gemini-powered assistant that suggests products from an uploaded image. Off by default; enabled via the shopping-assistant Kustomize component.
Cymbal branding mode	Optional	Re-skins the UI to 'Cymbal Shops' branding when CYMBAL_BRANDING=true on frontend.
Frontend banner message	Optional	Free-form banner shown at the top of the frontend when FRONTEND_MESSAGE is set.

2.1 What the product deliberately does NOT have

Useful to know because every one of these is a 'simplifying assumption' that a real-world change might violate, and the AI Change Analyzer should be alert to proposals that take the product into the territory of:

- No user accounts, signup, or login. Session is a UUID stored in the shop_session-id cookie.
- No persistent order history. PlaceOrder returns the OrderResult to the caller and emails the customer; nothing is stored long-term in any database.
- No inventory or stock tracking. Every product is effectively infinite. (RFC-2025-003 in the historical document proposes adding this.)

- No real payment processing. PaymentService validates credit card format and returns a mock transaction ID.
- No real email sending. EmailService logs the email content.
- No real shipping. ShippingService returns a mock tracking ID.
- No admin interface. Catalog is a static JSON file baked into the productcatalogservice container.
- No internationalization beyond currency. All UI text is in English.

3. User Journeys

Each journey below names every service that participates. The AI Change Analyzer uses these to expand 'this change affects service X' into 'this change affects user journeys A, B, and C.'

3.1 Browse home → product detail → add to cart

- frontend renders the home page
  - frontend → ProductCatalogService.ListProducts to fetch the 9 products
  - frontend → CurrencyService.Convert for every product price, in the user's selected currency
  - frontend → AdService.GetAds for the home-page ad slot
- User clicks a product; frontend renders the product detail page
  - frontend → ProductCatalogService.GetProduct
  - frontend → CurrencyService.Convert for the displayed price
  - frontend → RecommendationService.ListRecommendations passing the product ID
  - RecommendationService → ProductCatalogService.ListProducts (to filter against)
- User clicks Add to Cart
  - frontend → CartService.AddItem
  - CartService writes to redis-cart

3.2 View cart → enter checkout details → place order

- User opens the cart page
  - frontend → CartService.GetCart
  - frontend → ProductCatalogService.GetProduct for each cart item
  - frontend → CurrencyService.Convert for line items and total
  - frontend → ShippingService.GetQuote with a placeholder address
  - frontend → RecommendationService.ListRecommendations based on cart contents
- User submits the checkout form
  - frontend → CheckoutService.PlaceOrder with user_id, user_currency, address, email, credit_card
  - CheckoutService → CartService.GetCart
  - CheckoutService → ProductCatalogService.GetProduct for each item (to confirm pricing)
  - CheckoutService → CurrencyService.Convert as needed
  - CheckoutService → ShippingService.GetQuote then ShippingService.ShipOrder
  - CheckoutService → PaymentService.Charge
  - CheckoutService → EmailService.SendOrderConfirmation
  - CheckoutService → CartService.EmptyCart
  - CheckoutService returns OrderResult to frontend, which renders the confirmation page

3.3 Currency change

- User selects a new currency from the dropdown
    - frontend re-renders the current page with the new currency in the shop_session-id cookie's metadata; every price is reconverted via CurrencyService.Convert
    - This is why currencyservice is the highest-QPS service: a single page view can result in 10+ Convert calls, and the conversion happens on every page render

> **RISK PATTERN:** Any change that increases the number of CurrencyService.Convert calls per page render multiplies load on the highest-QPS service. The AI Change Analyzer should flag such changes for review by the owners of currencyservice.

4. Service Inventory

The complete current set of services with their languages, gRPC ports, and short descriptions. All ports are gRPC unless noted. Port numbers are taken from the kubernetes-manifests deployment files.

Service	Language	Port	Role
`frontend`	Go	`8080` (HTTP)	User-facing HTTP server. The only service exposed externally. Auto-generates a session ID for every visitor.
`cartservice`	C#	`7070`	Stores items in the user's cart, backed by Redis. Keyed by user_id (the session ID).
`productcatalogservice`	Go	`3550`	Serves the product catalog from a static JSON file. Supports ListProducts, GetProduct, SearchProducts.
`currencyservice`	Node.js	`7000`	Converts a Money amount from one currency to another. Highest-QPS service in the system.
`paymentservice`	Node.js	`50051`	Mock credit-card charge. Returns a generated transaction ID.
`shippingservice`	Go	`50051`	Mock shipping quote and order. Generates a tracking ID.
`emailservice`	Python	`8080` (gRPC)	Mock order-confirmation email. Logs the rendered email content.
`checkoutservice`	Go	`5050`	Orchestrates PlaceOrder: cart → catalog → currency → shipping → payment → email → empty cart.
`recommendationservice`	Python	`8080`	Returns up to 5 product IDs as recommendations, by sampling the catalog and excluding products already shown.
`adservice`	Java	`9555`	Returns up to 2 text ads given context keywords.
`loadgenerator`	Python (Locust)	–	Continuously hits frontend with realistic shopping flows. Not part of the production user path.
`redis-cart`	Redis (image)	`6379`	Backing store for cartservice. Default deployment is a single in-cluster pod, not Memorystore.

> **NOTE:** paymentservice and shippingservice both listen on port 50051 inside their own pods. They are distinguished by the Kubernetes Service name, not the port number. A change that moves either to a non-standard port must update the corresponding environment variable in every caller.

5. API Contracts (gRPC / Protocol Buffers)

All inter-service communication uses gRPC with the protobuf3 schema in protos/demo.proto. The proto package is hipstershop. Below is every RPC currently defined, grouped by service, with field-level detail. These are the contracts the AI Change Analyzer must reason about when assessing breaking changes.

> **Source:** protos/demo.proto, package hipstershop. The proto file is the single source of truth for all inter-service contracts; every language-specific stub is generated from it.

5.1 Shared Types

Money

Used everywhere a monetary value appears. Encodes a decimal value with nanosecond precision.

```
message Money {
  string currency_code = 1;  // ISO 4217 3-letter code
  int64 units = 2;          // whole units of the amount
  int32 nanos = 3;          // -999,999,999 to +999,999,999
}
// Example: $-1.75 → units=-1, nanos=-750,000,000
```

Sign rules: if units is positive, nanos must be positive or zero. If units is zero, nanos can be any sign. If units is negative, nanos must be negative or zero.

Address

```
message Address {
  string street_address = 1;
  string city = 2;
  string state = 3;
  string country = 4;
  int32 zip_code = 5;        // NB: integer, not string
}
```

> **KNOWN LIMITATION:** zip_code is an int32, which means it cannot represent postal codes with leading zeros (e.g. 02134 in Massachusetts) or alphanumeric codes (UK postcodes like SW1A 1AA). Any change that introduces real-world international shipping must change this field type, which is a breaking change.

CartItem

```
message CartItem {
  string product_id = 1;
  int32 quantity = 2;
}
```

5.2 CartService (port 7070)

```
service CartService {
  rpc AddItem(AddItemRequest) returns (Empty)
  rpc GetCart(GetCartRequest) returns (Cart)
  rpc EmptyCart(EmptyCartRequest) returns (Empty)
}

message AddItemRequest { string user_id = 1; CartItem item = 2; }
message GetCartRequest { string user_id = 1; }
message EmptyCartRequest { string user_id = 1; }
```

```
message Cart { string user_id = 1; repeated CartItem items = 2; }
```
user_id semantics: For anonymous shoppers, the session ID generated by frontend (stored in cookie shop_session-id) is used as user_id. There is no separate user concept.

5.3 ProductCatalogService (port 3550)

```
service ProductCatalogService {
  rpc ListProducts(Empty) returns (ListProductsResponse)
  rpc GetProduct(GetProductRequest) returns (Product)
  rpc SearchProducts(SearchProductsRequest) returns (SearchProductsResponse)
}

message Product {
  string id = 1;
  string name = 2;
  string description = 3;
  string picture = 4;       // relative URL path served by frontend
  Money price_usd = 5;      // always in USD; conversion is the frontend's job
  repeated string categories = 6;
}
```
Important: price_usd is always in USD. Conversion to the user's currency happens at the frontend layer via CurrencyService.Convert. Catalog stores USD only — see Section 6.

5.4 CurrencyService (port 7000)

```
service CurrencyService {
  rpc GetSupportedCurrencies(Empty) returns (GetSupportedCurrenciesResponse)
  rpc Convert(CurrencyConversionRequest) returns (Money)
}

message CurrencyConversionRequest {
  Money from = 1;
  string to_code = 2;       // ISO 4217 3-letter code
}
```
Rates source: Currency rates are read from a static rates table inside the service (originally sourced from European Central Bank). The base currency for the table is EUR; cross-rates are computed.

5.5 ShippingService (port 50051)

```
service ShippingService {
  rpc GetQuote(GetQuoteRequest) returns (GetQuoteResponse)
  rpc ShipOrder(ShipOrderRequest) returns (ShipOrderResponse)
}

message GetQuoteResponse { Money cost_usd = 1; }     // always USD
message ShipOrderResponse { string tracking_id = 1; } // mock; format: AAAA-9999
```

5.6 PaymentService (port 50051 inside its pod)

```
service PaymentService {
  rpc Charge(ChargeRequest) returns (ChargeResponse)
}

message CreditCardInfo {
  string credit_card_number = 1;
  int32 credit_card_cvv = 2;
  int32 credit_card_expiration_year = 3;
```

```
  int32 credit_card_expiration_month = 4;
}
message ChargeRequest { Money amount = 1; CreditCardInfo credit_card = 2; }
message ChargeResponse { string transaction_id = 1; }
```

PII & PCI NOTE: Credit card details flow through the system in plaintext gRPC messages. In a real deployment this would require PCI-DSS scope analysis. For the demo, no real card data is processed and the service rejects obviously invalid card numbers; valid-format cards return a UUID transaction ID.

5.7 EmailService

```
service EmailService {
  rpc SendOrderConfirmation(SendOrderConfirmationRequest) returns (Empty)
}

message OrderItem { CartItem item = 1; Money cost = 2; }
message OrderResult {
  string order_id = 1;
  string shipping_tracking_id = 2;
  Money shipping_cost = 3;
  Address shipping_address = 4;
  repeated OrderItem items = 5;
}
message SendOrderConfirmationRequest {
  string email = 1;
  OrderResult order = 2;
}
```

5.8 CheckoutService (port 5050)

```
service CheckoutService {
  rpc PlaceOrder(PlaceOrderRequest) returns (PlaceOrderResponse)
}

message PlaceOrderRequest {
  string user_id = 1;
  string user_currency = 2;
  Address address = 3;
  // field number 4 is reserved (was removed in the past)
  string email = 5;
  CreditCardInfo credit_card = 6;
}
message PlaceOrderResponse { OrderResult order = 1; }
```

PROTO FIELD HISTORY: PlaceOrderRequest skips field number 4 — it goes 1, 2, 3, 5, 6. This is the proto-level evidence of a past field removal. Field 4 is implicitly reserved and must never be reused; reusing a removed field number is a classic source of silent breakage in proto schemas. The AI Change Analyzer should flag any reuse of field number 4 in this message.

5.9 RecommendationService (port 8080)

```
service RecommendationService {
  rpc ListRecommendations(ListRecommendationsRequest) returns (ListRecommendationsResponse)
}

message ListRecommendationsRequest {
  string user_id = 1;
```

```
  repeated string product_ids = 2;   // products to exclude from results
}
message ListRecommendationsResponse {
  repeated string product_ids = 1;
}
```

Current behavior: Returns up to 5 random product IDs from the catalog, excluding those passed in product_ids. There is no actual ML model in the default deployment.

5.10 AdService (port 9555)

```
service AdService {
  rpc GetAds(AdRequest) returns (AdResponse)
}

message AdRequest { repeated string context_keys = 1; }
message Ad { string redirect_url = 1; string text = 2; }
message AdResponse { repeated Ad ads = 1; }
```

Current behavior: Returns up to 2 ads. If context_keys match a category, returns ads tagged with that category; otherwise returns random ads.

6. Data Models and Storage

6.1 Product catalog (static JSON)

The catalog is a static file baked into the productcatalogservice container at build time: src/productcatalogservice/products.json. There is no admin UI, no database, and no API for adding products. Adding a product requires editing the JSON and rebuilding the container image.

As of the current main branch, the catalog contains exactly 9 products:

Product ID	Name	Price (USD)	Categories
OLJCESPC7Z	Sunglasses	$19.99	accessories
66VCHSJNUP	Tank Top	$18.99	clothing, tops
1YMWWN1N4O	Watch	$109.99	accessories
L9ECAV7KIM	Loafers	$89.99	footwear
2ZYFJ3GM2N	Hairdryer	$24.99	hair, beauty
0PUK6V6EV0	Candle Holder	$18.99	decor, home
LS4PSXUNUM	Salt & Pepper Shakers	$18.49	kitchen
9SIQT8TOJO	Bamboo Glass Jar	$5.49	kitchen
6E92ZMYYFZ	Mug	$8.99	kitchen

Categories in use (closed set, derived from the catalog): accessories, clothing, tops, footwear, hair, beauty, decor, home, kitchen. Any code that hardcodes a category set must be checked against this list when categories change.

Price encoding in the JSON

```
{
  "id": "9SIQT8TOJO",
  "name": "Bamboo Glass Jar",
  "priceUsd": {
    "currencyCode": "USD",
    "units": 5,
    "nanos": 490000000
  },
  "categories": ["kitchen"]
}
```

Note the field is named priceUsd (camelCase) in the JSON, even though the proto field is price_usd (snake_case). This is the standard protobuf JSON mapping and applies to all snake_case fields when serialized.

6.2 Cart storage (Redis)

Carts are stored in Redis (key: cart:{user_id}, value: serialized Cart proto). The default deployment uses an in-cluster redis-cart pod backed by ephemeral storage; the kustomize components include alternatives backed by Memorystore, AlloyDB, and Cloud Spanner.

- Ephemeral: redis-cart pod with emptyDir volume (default). Cart data is lost on pod restart.
- Memorystore Redis: managed Redis (production-like). See kustomize/components/memorystore.
- AlloyDB: relational cart storage. See kustomize/components/alloydb.
- Spanner: global relational cart storage. See kustomize/components/spanner.

> **OPERATIONAL NOTE:** The default ephemeral deployment is the most fragile from a user-experience standpoint, but the most common in the demo's actual usage. Changes that assume cart persistence across pod restarts (e.g. abandoned cart emails) will not work with the default deployment.

6.3 Order and transaction data

There is no persistent order store. The flow is:

- CheckoutService assembles an OrderResult in memory during PlaceOrder
- PaymentService returns a fresh transaction_id (UUID)
- ShippingService returns a fresh tracking_id
- EmailService logs the order confirmation
- CheckoutService returns the OrderResult to the frontend, which renders the confirmation page
- CartService.EmptyCart is called to clear the user's cart
- No records of the order survive after the request completes

6.4 Session data

- Cookie name: shop_session-id
- Value: a generated session UUID, used as user_id throughout the system
- Special values reserved by the frontend liveness/readiness probes: shop_session-id=x-liveness-probe and shop_session-id=x-readiness-probe — any change to session handling must continue to honor these

6.5 PII surface

Data that would be classified as PII in a real deployment:

- email (passed to PlaceOrder, logged by EmailService)
- Address fields (street_address, city, state, country, zip_code)
- CreditCardInfo (number, CVV, expiration) — also PCI scope
- user_id (session UUID, not directly PII but a tracking identifier)

None of this is encrypted at rest in the default deployment — there is no 'at rest' because nothing is persisted server-side beyond the cart. In transit, encryption depends on whether Istio/Cloud Service Mesh is enabled (off in the base deployment).

7. Configuration Reference

Every environment variable that affects behavior. Variables are read at process start; changing them requires a pod restart. Variables marked 'inter-service' are the addresses each service uses to reach its dependencies.

7.1 frontend

Variable	Default	Effect
PORT	8080	HTTP listen port. Liveness/readiness probes hit /_healthz here.
PRODUCT_CATALOG_SERVICE_ADDR	productcatalogservice:3550	Inter-service. gRPC dial target.
CURRENCY_SERVICE_ADDR	currencyservice:7000	Inter-service.
CART_SERVICE_ADDR	cartservice:7070	Inter-service.
RECOMMENDATION_SERVICE_ADDR	recommendationservice:8080	Inter-service.
SHIPPING_SERVICE_ADDR	shippingservice:50051	Inter-service.
CHECKOUT_SERVICE_ADDR	checkoutservice:5050	Inter-service.
AD_SERVICE_ADDR	adservice:9555	Inter-service.
SHOPPING_ASSISTANT_SERVICE_ADDR	shoppingassistantservice:80	Inter-service. Only used when assistant is enabled.
CYMBAL_BRANDING	false	When true, re-skins the UI to 'Cymbal Shops' branding.
FRONTEND_MESSAGE	(empty)	Free-form banner string shown at the top of every page when non-empty.
ENABLE_ASSISTANT	false	When true, exposes the Gemini-powered shopping assistant UI.
ENABLE_PROFILER	0	When 1, enables Cloud Profiler.
ENV_PLATFORM	(unset)	Hint for which cloud platform the demo runs on; affects some UI text only.

7.2 productcatalogservice

Variable	Default	Effect
PORT	3550	gRPC listen port.

Variable	Default	Effect
EXTRA_LATENCY	(unset)	Injects a sleep of the given Go duration on every request (e.g. "5.5s"). Used for demonstrating latency-injection and chaos scenarios.
DISABLE_PROFILER	(unset)	When set, disables Cloud Profiler.
DISABLE_TRACING	(unset)	When set, disables OpenTelemetry tracing.

KNOWN QUIRK: productcatalogservice responds to UNIX signals: SIGUSR1 enables a 'dynamic catalog reload' feature that reloads products.json on every request, causing parseCatalog to dominate CPU time. SIGUSR2 disables it. This is a deliberately-bugged demonstration feature and should NEVER be enabled in benchmarking runs. See Section 10.

7.3 Observability variables (cross-service)

- COLLECTOR_SERVICE_ADDR — when set (typically to opentelemetrycollector:4317), services export traces to that OTLP endpoint
- ENABLE_TRACING — used in some services to gate tracing initialization
- ENABLE_STATS — appears in the google-cloud-operations Kustomize component but is not actually wired up in the service code (see issue #2399); it is effectively a no-op
- DISABLE_DEBUGGER — gates Cloud Debugger initialization on the services that support it

VERIFIED LIMITATION: ENABLE_STATS=1 is documented in some manifests but has no effect at runtime. Any change that assumes it does something is starting from a false premise. A code search for ENABLE_STATS in the source confirms no read of this variable.

7.4 Other notable per-service variables

- checkoutservice: addresses for each of its downstream dependencies (PRODUCT_CATALOG_SERVICE_ADDR, SHIPPING_SERVICE_ADDR, PAYMENT_SERVICE_ADDR, EMAIL_SERVICE_ADDR, CURRENCY_SERVICE_ADDR, CART_SERVICE_ADDR)
- recommendationservice: PRODUCT_CATALOG_SERVICE_ADDR; PORT
- cartservice: REDIS_ADDR (e.g. redis-cart:6379), PORT
- currencyservice: PORT, DISABLE_PROFILER, DISABLE_DEBUGGER, DISABLE_TRACING

7.5 Health probes (uniform across services)

HTTP services (frontend) use HTTP GET /_healthz on the service port. gRPC services use the standard gRPC health protocol on their service port. The frontend probe sends a special cookie shop_session-id=x-liveness-probe or x-readiness-probe so the request can be distinguished from real traffic and skipped from session-counting logic.

8. Business Rules and Behaviors

Rules that are encoded in code rather than configuration. Each is grounded in the implementation, not invented for the handbook.

8.1 Pricing and currency

- All catalog prices are stored in USD as a Money value (units + nanos)
- Conversion to display currency happens at the frontend just before rendering
- Shipping cost is quoted in USD by ShippingService and converted in the same way
- There is no rounding rule applied at conversion; the converted Money value retains nanos precision
- Cart totals are computed on the frontend by summing line items, then converting once at the end — not item-by-item, to avoid rounding drift

8.2 Cart rules

- A cart has no explicit size limit in code
- There is no per-item maximum quantity
- Adding the same product twice does NOT merge — each AddItem appends a new CartItem entry; quantity is set per call, not summed
- Cart persists as long as the Redis entry exists; the default ephemeral Redis loses carts on pod restart
- EmptyCart is called by CheckoutService after a successful PlaceOrder; there is no explicit cart-expiry logic

8.3 Checkout rules

- PlaceOrder is a single atomic-ish operation from the caller's perspective: success means the order was placed; failure means none of it took effect
- BUT: it is NOT a true atomic transaction. PaymentService is the source of truth for whether money moved. If EmailService fails AFTER payment succeeded, the order is still considered placed and the email is lost (no retry queue in the default deployment)
- Credit card validation: PaymentService checks the card number format and expiration; it rejects expired cards and obviously invalid card numbers
- There is no order_id collision check; order IDs are UUIDs

8.4 Recommendation rules

- Excludes products already in the request's product_ids list
- Returns at most 5 recommendations
- Selection is random sampling from the remaining catalog — not personalized
- If the catalog has fewer than 5 non-excluded products, returns whatever is available without padding

8.5 Ad rules

- Returns at most 2 ads per request
- If a context key matches a known ad category, returns an ad from that category
- Otherwise returns a random ad
- Ads have a redirect_url that is mock and not actually routed to anywhere meaningful in the demo

8.6 Currency rules

- Supported currency codes are the keys of the internal rates table; GetSupportedCurrencies returns this list
- If a Convert call uses a from currency code that is not in the table, the service returns an error
- Conversion: amount is normalized to EUR using the from rate, then to the target using the to rate (EUR is the pivot)
- Nano precision is preserved; no rounding to whole cents

9. Non-Functional Properties

What the system actually does for security, observability, and reliability today — distinct from what the historical document records as targets.

9.1 Authentication and authorization

- End users: NONE. The frontend assigns a session ID and trusts it.
- Service-to-service: NONE by default. gRPC calls are plaintext. mTLS is available only when deployed with Cloud Service Mesh / Istio via the kustomize service-mesh component
- Cluster-internal: relies entirely on Kubernetes NetworkPolicies or service mesh, neither of which is on in the base deployment

9.2 Encryption

- In transit (external): TLS is the responsibility of the ingress / load balancer in front of frontend; not configured in the base manifests
- In transit (internal): plaintext gRPC by default; mTLS with the service-mesh component
- At rest: not applicable for most services since nothing is persisted; the redis-cart deployment is in-memory only

9.3 Observability

- Tracing: OpenTelemetry traces exported to an OTLP collector when COLLECTOR_SERVICE_ADDR is set
- Metrics: per-service metrics emitted to the OTLP collector; some services have additional Cloud Operations exporters
- Logging: structured JSON logs to stdout/stderr; collected by the Kubernetes node agent
- Profiling: Cloud Profiler integrated in most services, gated by ENABLE_PROFILER or DISABLE_PROFILER (the polarity varies by service — a known inconsistency)

9.4 Reliability mechanisms

- Retries: gRPC client libraries have default retry policy on UNAVAILABLE; per-service retry tuning is minimal
- Timeouts: most callers use a default 5-second deadline; checkoutservice uses longer deadlines for downstream services
- Circuit breakers: none in the default deployment. Istio can add them when enabled.
- Bulkheading: each service runs in its own pod; replicas default to 1 in the base manifest, scaled horizontally via HPA where configured
- Graceful degradation: ad and recommendation services are best-effort; the frontend renders empty regions if they fail

9.5 Performance characteristics

- Highest QPS service: currencyservice (called many times per page render)
- Highest fan-out service: frontend on home/cart pages (4-7 downstream calls)
- Critical path service: checkoutservice (6 sequential downstream calls on PlaceOrder)
- Most CPU-intensive operation: ProductCatalogService.parseCatalog WHEN the dynamic-reload bug is enabled (>80% CPU). With the bug disabled, the service is essentially free.

9.6 Browser and client support

The frontend is server-rendered HTML with progressive enhancement via small amounts of JavaScript. It works on any modern browser. There are no native mobile apps in this repo — references to mobile apps elsewhere in this project's documentation (e.g. the historical document's INC-2024-07) are synthetic scenarios for capstone evaluation purposes only.

10. Known Quirks and Technical Debt

Real, verifiable issues in the current codebase. These are the items the AI Change Analyzer needs to know about because they affect how a change should be assessed.

10.1 productcatalogservice dynamic-reload bug (intentional)

- Toggle: SIGUSR1 enables the bug; SIGUSR2 disables it
- Behavior when enabled: products.json is re-read and re-parsed on every gRPC request
- Impact: parseCatalog dominates CPU; latency for ListProducts / GetProduct / SearchProducts grows sharply under load
- Purpose: this is a teaching feature for demonstrating profiling tools, not a real bug to be fixed
- AI Change Analyzer implication: any change to productcatalogservice that touches signal handling, parseCatalog, or the file-reload path must preserve this toggle behavior

10.2 productcatalogservice EXTRA_LATENCY env var

- Effect: injects a Go time.Duration sleep on every server call
- Purpose: latency injection for chaos / SLO demonstrations
- AI Change Analyzer implication: any change to the request path must keep the sleep in place

10.3 ENABLE_STATS no-op

- Source: github issue #2399
- Status: ENABLE_STATS is set in the google-cloud-operations Kustomize manifest patches but is not actually read by any service
- AI Change Analyzer implication: proposals that depend on toggling this variable are based on a false premise

10.4 ENABLE_PROFILER vs DISABLE_PROFILER polarity inconsistency

- Some services use ENABLE_PROFILER=1 to opt in; others use DISABLE_PROFILER (unset = on)
- AI Change Analyzer implication: changes that touch profiling configuration across services must check polarity per service

10.5 PlaceOrderRequest reserved field number 4

- As noted in Section 5.8, field 4 is skipped
- Reusing field 4 in this message would cause silent data corruption between old and new clients
- AI Change Analyzer implication: flag any proto change that adds a field with number 4 to PlaceOrderRequest as a HIGH-risk breaking change

10.6 zip_code as int32

- Address.zip_code is int32, which cannot hold leading-zero ZIPs or alphanumeric international postcodes
- AI Change Analyzer implication: any internationalization or 'real shipping' RFC must propose a breaking schema change; flag and require migration plan

10.7 No abandoned-cart durability

- Default deployment uses ephemeral Redis; cart data is lost on pod restart
- Any feature that relies on multi-session cart persistence (e.g. abandoned-cart emails, cross-device carts) requires changing the storage backend

10.8 No retry / DLQ between checkout and email

- If EmailService fails after PaymentService succeeds, the email is dropped
- AI Change Analyzer implication: any change that adds an additional post-payment step has the same risk profile and should be flagged

10.9 Single replica defaults

- The base kubernetes-manifests deploy each service with replicas: 1 by default
- Production-style availability requires the HPA / multi-replica components from kustomize

11. Optional Components and Deployment Variants

The kustomize directory contains components that can be layered onto the base deployment. The AI Change Analyzer needs to know these exist so it can correctly assess changes that target them.

11.1 shopping-assistant (Gemini AI)

- Adds the shoppingassistantservice (Python)
- Requires a Google Cloud API key for the Gemini model
- frontend exposes an upload-an-image UI when ENABLE_ASSISTANT=true
- Not present in the default deployment

11.2 Service mesh (Istio / Cloud Service Mesh)

- Injects sidecar proxies and enables mTLS between services
- Required for circuit breakers, retries with budgets, and authorization policies
- Adds significant complexity to the dependency graph (every service has a sidecar)

11.3 Alternative cart backends

- memorystore: managed Redis
- alloydb: PostgreSQL-compatible relational store
- spanner: globally distributed relational store
- Each requires changes to cartservice deployment manifests and (for AlloyDB/Spanner) to cartservice code paths

11.4 Observability components

- google-cloud-operations: enables Cloud Trace, Cloud Profiler, Cloud Logging exporters
- opentelemetry-collector: deploys a collector in-cluster

11.5 Loadgenerator

- Locust-based load generator that runs continuous synthetic shopping flows
- Not user-facing; deployed as a separate pod
- Generates the QPS that makes currencyservice the highest-traffic service

12. Cross-References to the Historical Document

How sections of this handbook line up with the Historical & Contextual Data document, so the AI system can pivot between current-state and history-aware reasoning.

Handbook Section	Historical Document Section
§3 User Journeys	§3 SLAs and Operational Baselines (esp. §3.1 Call Graph)
§4 Service Inventory	§2 Service Ownership Map
§5 API Contracts (esp. §5.8 PlaceOrderRequest field 4)	§5.2 INC-2024-07 (proto field removal incident)
§6 Data Models (§6.2 cart storage)	§5.3 INC-2024-11 (Redis failover incident); §6.1 RFC-2023-018 (cart store migration)
§7 Configuration Reference	§4 Service Runbooks (mitigation steps reference these variables)
§9 Non-Functional Properties	§3 SLAs and §6.3 RFC-2024-022 (gRPC upgrade)
§10 Known Quirks	§8 Anti-Patterns (each quirk maps to one or more patterns)

Appendix A — Source Validation

Every factual claim in this handbook traces to a specific source file in GoogleCloudPlatform/microservices-demo (branch main) or a verifiable upstream issue. The AI Change Analyzer can rely on these as ground-truth references when reasoning about changes.

Section	Source / verification
§2 Feature Inventory	README.md (service descriptions); kustomize/components/shopping-assistant/README.md (Gemini assistant); frontend.yaml env variables (CYMBAL_BRANDING, FRONTEND_MESSAGE, ENABLE_ASSISTANT).
§3 User Journeys	protos/demo.proto (RPC signatures showing which calls each flow involves); README.md.
§4 Service Inventory (ports, languages)	kubernetes-manifests/frontend.yaml (inter-service ADDR env values); README.md service table.
§5 API Contracts (all RPCs and message fields)	protos/demo.proto — entire file. PlaceOrderRequest field 4 gap directly visible in the proto source.
§6.1 Product catalog (9 products, prices, categories)	src/productcatalogservice/products.json — verified by reading the file.
§6.2 Cart storage backends	src/cartservice/ and kustomize/components/memorystore, alloydb, spanner directories.
§6.4 Session cookie name and probe values	kubernetes-manifests/frontend.yaml liveness/readiness probe Cookie header.
§7.1 frontend env variables	kubernetes-manifests/frontend.yaml env block — every variable cited appears there.
§7.2 productcatalogservice EXTRA_LATENCY and SIGUSR1/USR2	src/productcatalogservice/README.md sections 'Dynamic catalog reloading / artificial delay' and 'Latency injection'.
§7.3 ENABLE_STATS no-op	Upstream issue #2399 ('ENABLE_STATS doesn't do anything') — confirmed by maintainers as not wired up in code.
§8 Business rules	Inferred from proto definitions (e.g. Money sign rules from comments in demo.proto), products.json structure, and standard checkout flow described in the README and proto.
§9 Non-functional properties	Base kubernetes-manifests show no mTLS, no NetworkPolicies, no auth; service-mesh-istio Kustomize component README describes what changes when mesh is on.
§10 Known quirks	Each cross-referenced to its source: §10.1/10.2 → productcatalogservice/README.md; §10.3 → issue #2399; §10.5 → demo.proto line-by-line; §10.6 → demo.proto Address message.
§11 Optional components	kustomize/components/ subdirectory READMEs.

A.1 What was NOT directly verified

To be transparent about the limits of this handbook's grounding:

- Specific QPS numbers and 'highest-QPS service' claim — the repo README states currencyservice is 'the highest QPS service' but does not give absolute numbers. The numeric QPS values appear only in the synthetic historical document.
- Recommendation algorithm details (random sampling, max 5) — derived from the proto signature and common implementation patterns; the actual src/recommendationservice/recommendation_server.py would confirm exactly, but the high-level shape is consistent with the response message structure.
- Ad selection details (max 2, category match) — derived from the proto and adservice source patterns.
- Currency rates pivot through EUR — consistent with how the original currencyservice ECB feed worked; confirmable in src/currencyservice/server.js.

A.2 Maintenance notes

- When the proto file changes, regenerate §5 from the new demo.proto.
- When products.json changes, regenerate the table in §6.1.
- When kubernetes-manifests env blocks change, regenerate §7.
- Section §10 should grow as new quirks are discovered — but only after a code-level or issue-level verification.